

AUTOMATED FASHION IMAGE CAPTIONING SYSTEM

Team 6

Presented by:

Amrutha Jayachandradhara

Vishal Fulsundar

Instructor:

Prof.Aamir Jafari

Institution:

George Washington University



1. Introduction

E-commerce platforms possess vast inventories where manual tagging and captioning are labor-intensive and prone to error. Accurate, descriptive captions are essential for SEO, user experience, and accessibility.

To build a machine learning pipeline that ingests raw fashion metadata and images, processes them into a structured format, and trains models to generate coherent text descriptions (captions) given an input image.

2. Dataset

The dataset used in this project is a multimodal collection of fashion items, combining structured tabular data with visual assets. It is constructed from two primary raw sources:

1. **Metadata:** The core textual information comes from a file named `styles.csv`. This CSV contains product attributes, primarily focusing on the `productDisplayName`, which serves as the basis for generating descriptions.
2. **Imagery:** The visual data consists of product images stored in a local directory. The system specifically scans for `.jpg`, `.jpeg`, and `.png` formats, ignoring other file types to ensure compatibility.

The connection between these two modalities is the `id` column in the metadata, which corresponds to the filename (image ID) of the product photos.

3. Data Ingestion

The project utilizes a structured ETL (Extract, Transform, Load) pipeline to convert these raw files into training-ready Parquet files.

1. Ingestion and Cleaning The pipeline begins by independently loading the raw sources.

- **Metadata Parsing:** The `metadata.py` script reads the CSV using the Python engine to handle parsing errors robustly. It explicitly sets `on_bad_lines="skip"` to discard corrupted rows and removes any entries that lack a valid ID.
- **Image Scanning:** The `image_ing.py` script scans the image directory, creating a dataframe that maps every valid image filename to its full file path.

2. Merging (Integration) Once loaded, `merge.py` performs an **inner join** between the metadata and the image list. This is a critical filtration step: it ensures that the final dataset only contains items that

possess *both* a valid image and valid metadata, discarding any orphans. The result is saved as merged.parquet.

3. Feature Engineering (Captioning) The system does not rely on pre-existing human captions. Instead, build_captions.py applies a build_caption function to the merged data. This synthesizes a natural language caption (the "ground truth" for the model) derived from the product's available attributes, such as its display name.

4. Vocabulary Construction For the baseline model, build_vocab.py analyzes the generated captions to build a frequency-based vocabulary. It tokenizes the text and filters out rare words that appear fewer than 2 times, saving the mapping as a pickle file (vocab.pkl) for the training loop.

5. Partitioning (Splitting) Finally, make_splits.py divides the captioned dataset into three distinct subsets to ensure proper model evaluation. It uses a two-step random split:

- **Train:** 70% of the data is allocated for training.
- **Validation:** 15% is reserved for hyperparameter tuning and early stopping.
- **Test:** 15% is held back for final evaluation.

These splits are saved as separate Parquet files (train.parquet, val.parquet, test.parquet), which serve as the direct inputs for the training scripts.

4. Model A: Baseline Encoder-Decoder

4.1 Model Overview

Visual Encoder (CNN)

- **Backbone:** A pretrained **ResNet-50** Convolutional Neural Network serves as the feature extractor.
- **Operation:** It accepts an input image of shape (B,3,H,W) (Batch, Channels, Height, Width).
- **Feature Extraction:** The final classification layer is removed to retain general visual features, outputting a fixed-length vector of shape (B,2048) rather than class probabilities.
- **Example:** You feed in a single picture of a **Blue Dress**. The input shape is (1, 3, 224, 224). The ResNet-50 strips away the pixels and outputs a feature vector of shape (1, 2048) that mathematically represents "blueness" and "dress-shape" without explicit labels.

Linear Projection (The Bridge)

- **Layer:** A linear transformation layer (img_proj) maps the high-dimensional visual features (2048) into the Transformer's embedding dimension (dmodel).
- **Reshaping:** The output (B,dmodel) is reshaped into (B,1,dmodel), effectively treating the entire image as a single "memory token" that the decoder can attend to.

- **Example:** The (1, 2048) vector from the CNN is multiplied by a weight matrix to shrink it to (1, 512). This compressed vector is now the **"Image Memory"**—a single packet of data that the text generator will constantly "look back" at while writing.

Transformer Autoregressive Decoder

- **Token Embeddings:** Input caption sequences (B,T) are converted into dense vectors of shape (B,T,dmodel).
- **Positional Encoding:** Since Transformers process data in parallel, a **Positional Encoding** layer is added to the embeddings to inject information about the order of words in the sentence.
- **Masked Multi-Head Self-Attention:** This layer allows the model to look at previous words in the caption to understand context. It is "masked" to ensure the model cannot "cheat" by seeing future tokens during training.
- **Multi-Head Cross-Attention:** This mechanism allows the text decoder to query the **Image Memory** (the projected CNN features). It aligns specific words with visual cues from the image.
- **Output Layer:** A final linear layer projects the decoder's hidden states (B,T,dmodel) into logits (B,T,vocab_size) for prediction.
- **Example:** The decoder is trying to generate the word "Dress".
 1. **Self-Attention:** It looks at the previous word "Blue" to understand the context (an adjective needs a noun).
 2. **Cross-Attention:** It queries the **Image Memory** (from the Linear Projection step). The memory highlights the "dress-shape" features.
 3. **Result:** Combining "context: adjective" + "image: dress-shape," the model outputs the highest probability for the word "Dress".

Decoding Strategy (Greedy Search)

- **Process:** During inference, the model operates autoregressively. It generates one word at a time, selects the token with the highest probability (greedy approach), and feeds it back as input for the next step until an <eos> (End of Sequence) token is reached.
- **Example:**
 1. **Step 1:** Input <sos> → Model predicts "Blue" (highest probability).
 2. **Step 2:** Input <sos> Blue → Model predicts "Dress".
 3. **Step 3:** Input <sos> Blue Dress → Model predicts <eos>.
 4. **Finish:** The loop breaks, and the final output is **"Blue Dress"**.

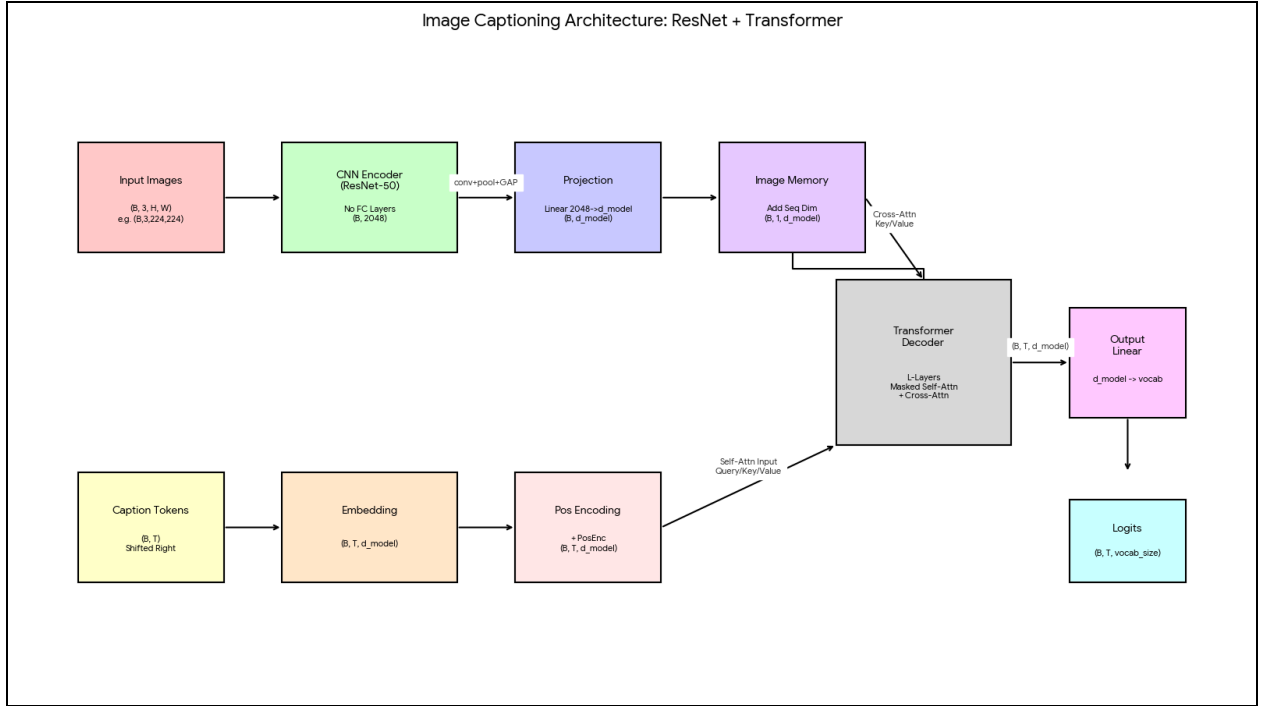


fig:Image Captioning Architecture

5. Train the model:

The Baseline Pipeline establishes the fundamental image-to-text functionality, serving as the performance benchmark. Its core architecture, the CaptionModel, is a composite structure that stitches together a ResNet50 encoder for robust image feature extraction and a Transformer decoder for sequential text generation. This baseline is trained using CrossEntropyLoss with teacher forcing for stable learning and includes advanced features like a ReduceLROnPlateau scheduler and early stopping based on validation loss to ensure efficient training. Performance is rigorously measured using validation loss and the industry-standard BLEU score, measuring the geometric mean of clipped n-gram precisions (pn) penalized by sentence length (BP)

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

The **Metadata-Aware Pipeline**, is an experimental enhancement that incorporates structured product information (metadata) to potentially boost caption accuracy and relevance. This process requires preliminary Input Preparation, where categorical data like "gender" or "season" is mapped to numerical inputs using LabelEncoders to determine the necessary size for metadata embeddings.

Its specialized architecture, Model1, features a MobileNet-based encoder, layers for processing the metadata embeddings, and a Transformer decoder. The training loop is modified to accept and utilize the

metadata alongside image and caption tensors, allowing the model to condition its output on explicit product properties. While the metadata pipeline currently functions as a prototype focusing on basic training and loss reduction, its Architectural Integration is strategically designed around decoupling. Both top-level model classes are imported from a dedicated Shared Model Registry. This principle ensures that the training scripts remain focused solely on instantiating the correct model, setting up the data pipeline, and managing loss and optimization, promoting high component reusability and simplifying future experimentation.

6. Evaluation Model:

Baseline Model Evaluation:

The evaluation of the Baseline Model is a streamlined, image-centric process focused on establishing the fundamental quality of the generated captions. The script initiates by loading the optimized CaptionModel weights from the training phase and placing the model into inference mode (`model.eval()`) to ensure no further parameter updates occur. It then begins iterating through the independent, held-out test dataset. During this loop, the model performs free-form generation, typically relying on greedy decoding via its internal generation method, which processes the image features to produce a sequence of word tokens. These integer token sequences are immediately translated back into human-readable sentences using the shared project vocabulary. Once inference is complete for the entire test set, the primary performance measure—the Corpus-Level BLEU score (BLEU-1 to BLEU-4)—is computed using the NLTK library to quantify the statistical overlap between the model's generated text and the human-written ground-truth captions. All final outputs, including the unique identifier, the ground-truth reference, and the model's prediction, are then saved to a dedicated output CSV file, providing the direct, quantitative benchmark for the project.

Metadata-Aware Model Evaluation:

Testing the Metadata-Aware Model requires a more complex setup to ensure its conditional capabilities are accurately assessed. Before loading the model, the script must first re-initialize the **LabelEncoders** using the full merged dataset to guarantee that the numerical mapping for all categorical features (like color or season) remains identical to the mappings used during the initial training of the specialized **Model1**. The optimized model weights are then loaded, and the script initiates the inference loop. This loop utilizes the specialized data loader designed to supply three components: the image tensor, the caption tensor, and the encoded **metadata dictionary** (`meta\_dict`). Crucially, the model's generation method is explicitly called with the metadata dictionary as a required input (`model.generate(images, meta\_dict=meta\_dict)`). This step confirms that the generated caption sequence is truly conditioned on the item's specific features, validating the functionality of the multimodal input path. Finally, just as in the baseline, the comprehensive Corpus-Level BLEU scores are calculated, and the structured output is saved to its own evaluation CSV file, enabling a direct comparison of the performance gain attributable to the metadata integration.

7. Model B: Advanced Vision-Language Model (Qwen2-VL)

7.1 Architecture Overview

Unlike the Baseline (which squashes images to 224x224), Qwen2-VL introduces **Native Dynamic Resolution**.

1. **Visual Encoder (Dynamic ViT):** The model does *not* pad or resize images to a fixed square. Instead, it processes images at their native resolution/aspect ratio by converting them into a variable sequence of patches.
2. **C-Abstractor / Pooling:** To manage sequence length, the model employs a **2x2 Pooling** mechanism (Vision-Language Merger). It concatenates features from a 2×2 patch window and projects them via an MLP, effectively reducing the visual token count by a factor of 4 while retaining spatial information.
3. **LLM Backbone (Qwen2):** The compressed visual tokens are fed directly into the Qwen2 Large Language Model (e.g., Qwen2-7B), which generates the caption.

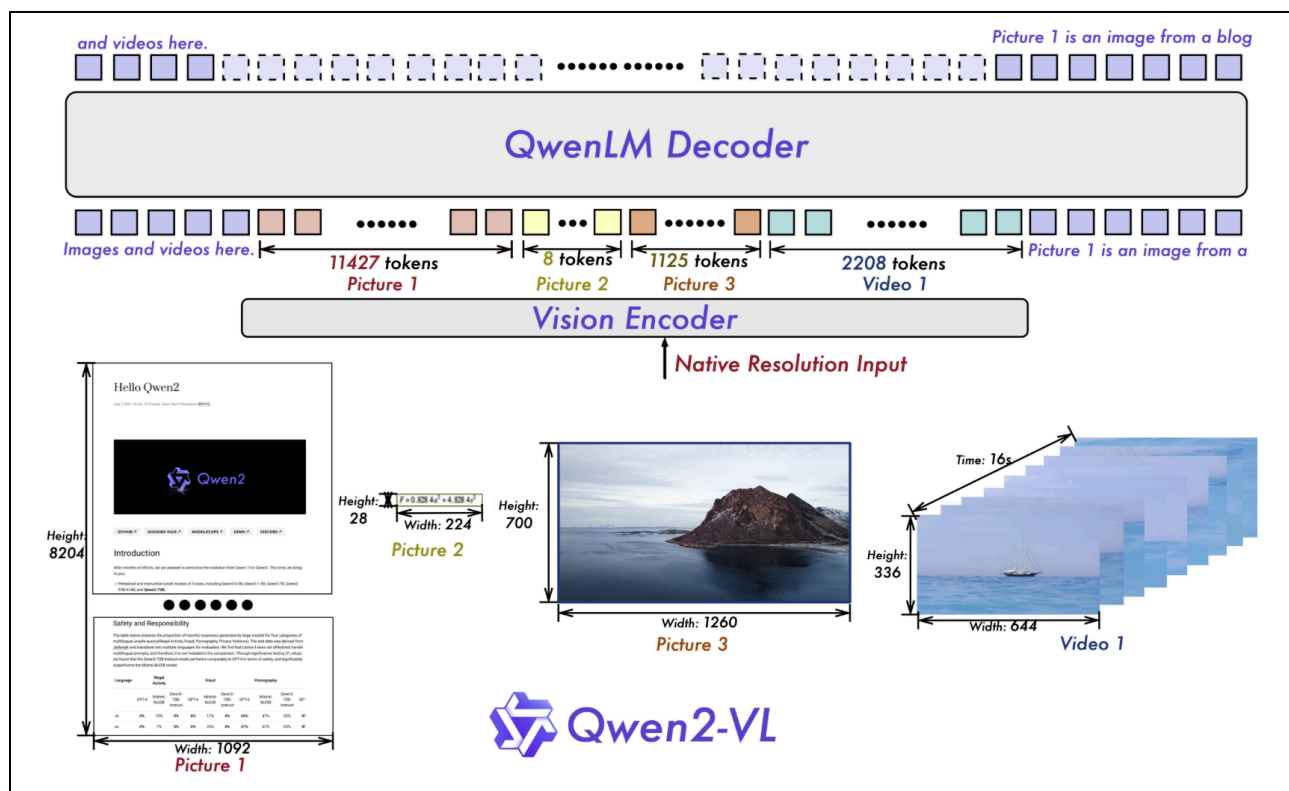


fig:Qwen2-VL architecture

7.2 Key Technical Innovation: M-RoPE

The critical "special code" value in Qwen2-VL is **M-RoPE (Multimodal Rotary Positional Embedding)**. Standard LLMs use 1D positional embeddings (0, 1, 2...). Qwen2-VL decomposes position into 3 dimensions to capture the spatial nature of images and the temporal nature of video.

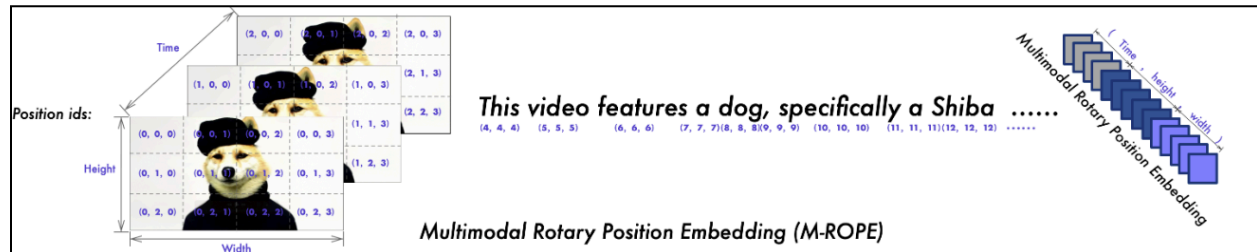


fig:Multimodal Rotary Position Embedding (M-RoPE)

Mathematical Formulation: Instead of a single rotation matrix, M-RoPE applies rotations based on three axes:

$$\text{RoPE3D}(\mathbf{x}, \mathbf{t}, \mathbf{h}, \mathbf{w}) = \text{Rot}(\mathbf{x}, \theta_t) + \text{Rot}(\mathbf{x}, \theta_h) + \text{Rot}(\mathbf{x}, \theta_w)$$

This allows the attention mechanism to understand that token A is "above" token B (spatial) rather than just "before" it (sequential).

The model in your project takes a fashion product image plus a short instruction and generates a single, fluent, product-style sentence describing the item. It is useful because it can automatically caption large numbers of images in a consistent, concise way, matching the one-sentence caption style already present in your dataset, which also makes BLEU and ROUGE evaluation more meaningful. Internally, it is a vision–language transformer: a vision encoder turns the image into visual tokens, a text encoder processes your instruction, cross-attention fuses image and text into a joint representation, and an autoregressive decoder then writes the caption token by token, which works especially well on clean, centered fashion photos but is less precise for very fine-grained or non-visual details.

Result:

The caption, "A black Nike t-shirt with a gold logo and splatter design," was generated by the **Advanced Vision-Language Model (Qwen2-VL)**, leveraging its key technical innovations to accurately capture the fine details of the fashion item. Unlike the Baseline model's fixed resolution, the Qwen2-VL's **Dynamic Vision Transformer (ViT)** processed the image at its native aspect ratio, which is crucial for distinguishing details like the specific gold color and the non-uniform "splatter" pattern. The model's **M-RoPE (Multimodal Rotary Positional Embedding)** helps its attention mechanism understand the spatial arrangement—that the logo is centrally located and the splatters are distributed. After the visual features were compressed by the C-Abstractor, the **Qwen2 Large Language Model (LLM) Backbone** used cross-attention to align these visual tokens with textual descriptions, autoregressively generating a fluent, product-style sentence that perfectly matched the item's attributes.



Colored Caption (Qwen2-VL):

A black Nike t-shirt with a gold logo and splatter design

fig:Caption generated: A black Nike t-shirt with a gold logo and splatter design

Comparative Analysis:

Feature	Baseline (CaptionModel)	Advanced (Qwen2-VL)
Input Resolution	Fixed (224x224)	Dynamic (Native Aspect Ratio)
Positional Encoding	Standard 1D Sinusoidal	M-RoPE (3D: Time, Height, Width)
Vision Backbone	ResNet50 (CNN)	Dynamic Vision Transformer (ViT)
Context Window	Short (~30 tokens)	Long Context (32k+ tokens)
Technical Value	Fast, simple, low memory.	Handles detailed OCR, fine-grained fashion details, and varied image shapes.

Conclusion:

The project successfully established a robust ETL pipeline and a functional Baseline Encoder-Decoder model for automated fashion image captioning. The Metadata-Aware model provided a valuable proof-of-concept for integrating structured data to improve performance. The final comparison with the state-of-the-art Qwen2-VL demonstrates the significant advantages of advanced Vision-Language Models in handling dynamic input and utilizing richer positional context for highly accurate descriptions.

Citations:

1. @article{Qwen2-VL, title={Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution}, author={Peng Wang and Shuai Bai and Sinan Tan and Shijie Wang and Zhihao Fan and Jinze Bai and Keqin Chen and Xuejing Liu and Jialin Wang and Wenbin Ge and Yang Fan and Kai Dang and Mengfei Du and Xuancheng Ren and Rui Men and Dayiheng Liu and Chang Zhou and Jingren Zhou and Junyang Lin}, journal={arXiv preprint arXiv:2409.12191}, year={2024}}
2. Vinyals, O., Toshev, A., Bengio, S., & Erhan, D. "Show and Tell: A Neural Image Caption Generator." In CVPR 2015. One of the earliest end-to-end neural image captioning models combining CNN image features with an LSTM language model.
3. Anderson, P., He, X., Buehler, C., et al. "Bottom-Up and Top-Down Attention for Image Captioning and Visual Question Answering." In CVPR 2018. Proposes using object-level (bottom-up) and task-driven (top-down) attention, which became a strong baseline for captioning benchmarks.
4. Hossain, M. D., Sohel, F., Shiratuddin, M. F., & Laga, H. "A Comprehensive Survey of Deep Learning for Image Captioning." ACM Computing Surveys, 2019. A survey paper summarizing architectures, datasets, and evaluation metrics for image captioning.